Step 8 — Merge, export, soglia e congelamento

Obiettivo

Trasformare l'adapter scelto in un **modello pronto all'uso**: pesi fusi, formato di produzione (per Winnow, GGUF per il suo server), soglia e temperatura definitive, verifica che le probabilità non siano cambiate nel passaggio, congelamento della configurazione e valutazione finale sul test.

Input e output

Input	adapter del checkpoint scelto, pesi di partenza, `dev_select` e `dev_confirm` dello Step 0
Output	`finetune/release/<modello>-fc-<versione>/`: pesi fusi bf16, (Winnow) GGUF BF16 e Q8_0, `decision.json` (soglia, temperatura, domande), `frozen.json`, report finale
Codice	`finetune/merge.py` (**da scrivere**)

1. Fusione dell'adapter

LoRA aggiunge $W' = W + (\alpha/r)\cdot B\cdot A$. Fondere significa calcolare W' una volta e salvare pesi normali: in inferenza non serve più PEFT e la velocità è quella del modello originale.

```
from peft import PeftModel
from transformers import AutoModelForCausalLM
base = AutoModelForCausalLM.from_pretrained(BASE_DIR, torch_dtype="bfloat16")
model = PeftModel.from_pretrained(base, ADAPTER_DIR).merge_and_unload()
model.save_pretrained(OUT_DIR, safe_serialization=True)
# copiare anche tokenizer, chat template e config, identici a quelli di partenza
```

Attenzioni:

- fondere sempre su pesi **bf16**, anche se il training era QLoRA (4 bit): fondere su pesi quantizzati degrada il modello;
- per JPT copiare i file della revisione `b447cc7` (tokenizer, template); per Winnow quelli di `winnow-12b-bf16/model/` (config `gemma4_text`, tokenizer, `chat_template.jinja`).

2. Verifica dopo la fusione

Su 200 righe di `dev_select` (5 fonti), confrontare le probabilità del modello **con adapter** (PEFT) e del modello **fuso** (vLLM, come nei benchmark):

Controllo	Soglia
Differenza media di P(supportato)	$< 0{,}005$
Decisioni diverse	0–1 su 200

Se le differenze sono maggiori, c'è un errore nella fusione o nei file copiati.

3. Winnow: ritorno al formato GGUF

Winnow in produzione gira sul suo server llama.cpp (`winnow-12b/serve.sh`), che legge solo GGUF. Passi:

1. Convertire i pesi fusi in GGUF con il convertitore di llama.cpp (la stessa versione del fork usato dal server, scaricato da `winnow-12b/setup.sh`), prima in **BF16**, poi quantizzare in **Q8_0** (ed eventualmente NVFP4).
2. Il server di Winnow verifica l'hash atteso del modello: per un modello nuovo va avviato indicando la cartella con il nuovo GGUF (`MODEL_DIR=... winnow-12b/serve.sh`); controllare nella documentazione del server (`winnow-12b/src/docs/`) come registrare un modello diverso.

3. **Ripetere il controllo già fatto per la conversione originale**: 63 righe di test, probabilità del server (GGUF) contro vLLM (bf16). Riferimento della conversione originale: differenza media 0,007 / 0,019 / 0,015 per le tre domande con Q8_0; 62 decisioni uguali su 63.

Formato	Uso	Atteso rispetto a bf16
BF16 GGUF	riferimento sul server	praticamente identico
Q8_0	produzione consigliata	differenze piccole (come sopra)
NVFP4	GPU con poca memoria	differenze maggiori (originale: 0,04–0,08 medie, 58/63 decisioni uguali)

Per JPT il formato di produzione resta safetensors con vLLM (come in `jpt-9b/benchmark.py`).

4. Soglia e temperatura finali

Dopo il fine-tuning le probabilità sono cambiate: soglia e temperatura della baseline non valgono più.

1. Inferenza su `dev_select` e `dev_confirm` con il modello fuso, nel formato di produzione.
2. Fit della **temperatura** (minimizzando il log-loss), poi della **soglia** (massimizzando la BAcc macro, griglia 0,01–0,99), come fa `calibrate_transfer.py fit`.
3. Conferma su `dev_confirm`: la BAcc non deve scendere di più di ~1 punto rispetto a `dev_select`.
4. Soglia **secondaria**, dichiarata: fittata sul dev di LLM-AggreFact (pool `tune` + `selection`, 4.520 righe) e applicata solo al test di AggreFact, per confronto con i numeri già prodotti.

Esempio di `decision.json`:

```
{"model": "winnow-12b-fc-v1", "format": "gguf-q8_0",
 "questions": "support_simple", "rule": "p_support > threshold",
 "temperature": 1.15, "threshold": 0.46,
 "fitted_on": "llm-aggrefact dev / tune pool", "confirmed_on": "selection pool"}
```

(I numeri sono un esempio di formato, non un risultato.)

5. Congelamento

Prima di guardare il test, salvare `frozen.json` con:

- hash SHA-256 dei pesi fusi (o del GGUF), del tokenizer e del template;
- hash del mix e del dataset renderizzato usati per il training;
- configurazione di training ed esperimento di provenienza;
- `decision.json`;
- hash del codice di valutazione.

È lo stesso schema usato per il risultato di Jev in questa repo (`results/frozen.json`): dopo il congelamento, nessuna modifica in risposta ai risultati del test.

6. Valutazione finale

Una sola esecuzione dei blocchi B e D di 06_valutazione.md sulla configurazione congelata, con il report nel formato di §7 di quel file e la tabella LLM-AggreFact per fonte nel formato della leaderboard. Riportare accanto: baseline, controllo C1 (modello base fine-tunato), Bespoke-MiniCheck-7B.

7. Pacchetto finale

File	Contenuto
pesi (safetensors e/o GGUF)	modello fuso
`decision.json`	come usarlo: domanda, regola, soglia, temperatura
`frozen.json`	hash e provenienza

File	Contenuto
`REPORT.md`	risultati, limiti, dati usati, licenze
`MODEL_CARD.md`	uso previsto (fact checking documento–claim, inglese), cosa non fa (verità nel mondo reale senza documento, retrieval)

Licenze. JPT-9B è CC BY-NC 4.0: anche il modello derivato è non commerciale. Per Winnow (Apache 2.0) la licenza del derivato dipende dai dati: ANLI, FactKG, AVeriTeC e SciFact sono non commerciali. Un Winnow commerciale va addestrato solo con dati a licenza permissiva (MiniCheck MIT, SummEdits CC BY, HoVer/FEVER/VitaminC CC BY-SA, dati generati con modelli aperti) e senza output di API i cui termini lo vietano.

Criteri di completamento

- Verifica della fusione e (per Winnow) della conversione GGUF superate.
- Soglia e temperatura fittate su `dev_select` e confermate su `dev_confirm` (più la soglia secondaria di AggreFact, dichiarata).
- `frozen.json` scritto prima del test; test eseguito una volta; report pubblicato.